

# Senior Full Stack Engineer

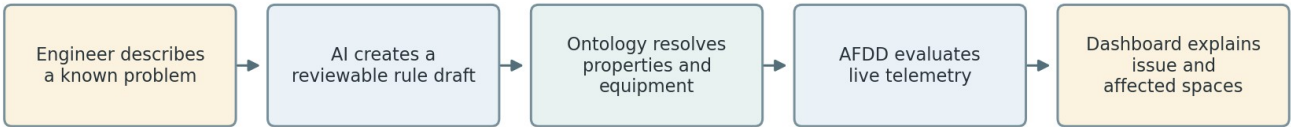
## Technical Assessment

*Technical ownership of a configurable multi-site AFDD product*

**Your mission**

Act as the technical owner of a new building-operations capability. Design and build a working product that helps property engineers convert recurring operational know-how into safe, reusable Automatic Fault Detection and Diagnostics (AFDD) monitoring.

The product should answer two practical questions: What equipment needs attention, and why? A user should also understand which occupied spaces may be affected and how the rule can be improved.



*One connected product journey*

**The demonstration finish line**

An engineer describes the supplied AHU problem in everyday language, reviews the system interpretation and matched assets, activates the rule, replays telemetry, observes the seeded fault, and investigates its evidence and affected tenant rooms.

**Four connected tasks**

| Task                        | Focus                                                                        |
|-----------------------------|------------------------------------------------------------------------------|
| 1 Foundation                | Stream and organize telemetry from multiple buildings with ontology context. |
| 2 Configurable AFDD engine  | Target and evaluate one reusable AHU rule with traceable evidence.           |
| 3 Operation Dashboard       | Explain system health, detected issues, affected spaces, and rule behavior.  |
| 4 AI-assisted rule creation | Turn natural-language requests into safe, reviewable rule drafts.            |

*Expected effort: approximately 16-20 hours across 4-5 days with AI coding agents. This is an estimate, not a time cap. You own the outcome and should communicate material scope decisions.*

# 1 Building operations primer

No property-management or HVAC background is assumed. This primer defines the concepts needed for the assessment; the supporting pack provides the authoritative model and data.

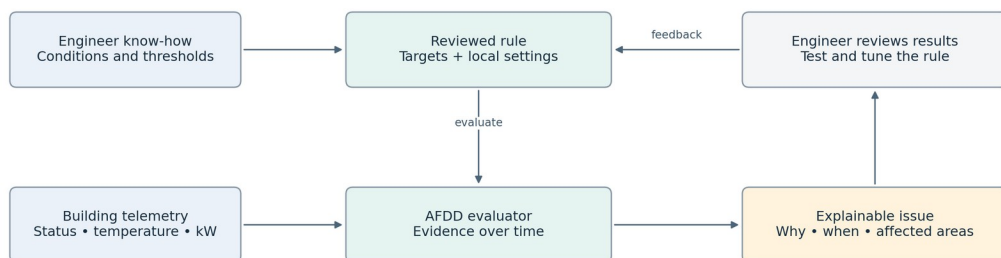
## Who uses the product

| User               | What they know                                           | What the product should help them do                                 |
|--------------------|----------------------------------------------------------|----------------------------------------------------------------------|
| Property engineer  | Recurring operating problems and local building behavior | Turn know-how into reusable monitoring rules and tune local settings |
| Technician         | How to investigate equipment and on-site conditions      | Understand why an issue triggered and which spaces may be affected   |
| Operations manager | Portfolio priorities and service performance             | See where attention is needed and whether monitoring is reliable     |

## HVAC and AFDD in this exercise

HVAC means heating, ventilation, and air conditioning. An air handling unit (AHU) moves and conditions air for an area. Supply-air temperature is the measured air leaving the AHU; the supply-air temperature setpoint is its target. Return-air temperature is the air coming back from the served spaces.

Automatic Fault Detection and Diagnostics (AFDD) continuously evaluates operating conditions and recent telemetry. In this assessment, diagnostics means preserving enough evidence to explain why a condition was detected. An AFDD issue is a reason to investigate, not proof of a mechanical root cause.



*The AFDD loop turns live readings and engineering rules into explainable issues and better future rules.*

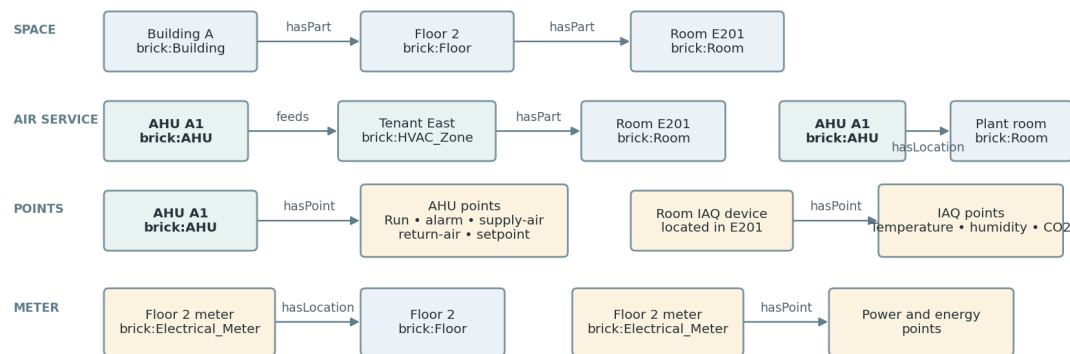
## Devices and telemetry

| Asset                     | Provided telemetry                                                             | Role in this assessment                     |
|---------------------------|--------------------------------------------------------------------------------|---------------------------------------------|
| AHU                       | Run status, alarm, supply-air and return-air temperatures, supply-air setpoint | Required AFDD input plus diagnostic context |
| Electricity meter         | Electrical power in kW and cumulative energy in kWh                            | Floor-level operational context             |
| Indoor air quality device | Room temperature, humidity, and CO2                                            | Context for potentially affected rooms      |

## 2 Ontology and spatial understanding

### Why the system needs shared meaning

Every building uses different names and identifiers. An ontology gives those site-specific entities shared types and relationships. Brickschema is an open ontology for describing building spaces, equipment, telemetry points, and how they relate. A graph database may store those relationships, but Brickschema is the semantic model, not the database itself. [1]



Independent lanes show space containment, AHU service and location, and device ownership of telemetry points.

### Required spatial meaning

- A building contains floors and rooms. An HVAC zone groups the rooms that receive air together.
- Each AHU is installed in the plant room and directly serves one HVAC zone. Its physical position is different from its served-space and potentially affected-space path.
- Each AHU, IAQ device, and electricity meter owns telemetry points. A floor meter measures the supplied floor scope; it is not an AHU-specific meter.
- A rule can select equipment by property type, served floor, served zone or room, and equipment type. Selection must follow explicit relationships rather than infer meaning from names.

### Reference classes

| Group              | Brickschema reference classes                                                                                                                  |
|--------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| Spaces             | brick:Building, brick:Floor, brick:Room, brick:HVAC_Zone                                                                                       |
| Equipment          | brick:AHU, brick:Electrical_Meter                                                                                                              |
| AHU points         | brick:Run_Status, brick:Alarm, brick:Supply_Air_Temperature_Sensor, brick:Return_Air_Temperature_Sensor, brick:Supply_Air_Temperature_Setpoint |
| Other measurements | brick:Electrical_Power_Sensor, brick:Electrical_Energy_Sensor, brick:Zone_Air_Temperature_Sensor, brick:Humidity_Sensor, brick:CO2_Sensor      |

Use the Brickschema version and mappings pinned in the supporting pack. References: [1] [docs.brickschema.org/intro.html](https://docs.brickschema.org/intro.html) [2] [docs.brickschema.org/brick/relationships.html](https://docs.brickschema.org/brick/relationships.html)

### 3 Assessment data and scope

The supporting pack removes the need to invent building data. It is the source of truth for entity IDs, relationships, units, timestamps, data-quality values, and expected outcomes.



Buildings A and B are offices with tenant rooms; Building C is a hotel with guest rooms. Each floor has two zones, and each zone has two occupied rooms.

#### Repeatable three-building inventory

- Each building has four floors, one lobby, one plant room, eight AHUs, four floor electricity meters, and sixteen occupied rooms.
- One AHU directly serves one zone. Each zone contains two rooms. Each occupied room has one IAQ device with temperature, humidity, and CO2 points.
- Across the portfolio: 3 buildings, 12 floors, 24 zones, 48 occupied rooms, 24 AHUs, 48 IAQ devices, and 12 meters.

#### What AltoTech provides

| Asset                     | Purpose                                                                                     |
|---------------------------|---------------------------------------------------------------------------------------------|
| Ontology and inventory    | Authoritative entities, types, relationships, property/space metadata, and expected counts  |
| Telemetry fixtures        | Ordered normal, sustained-fault, OFF, missing-data, recovery, duplicate, and invalid events |
| Rule verification cases   | Expected target sets, exclusions, issue windows, and one property-specific override         |
| Natural-language requests | Supported, paraphrased, ambiguous, unsupported, no-match, and failure cases                 |

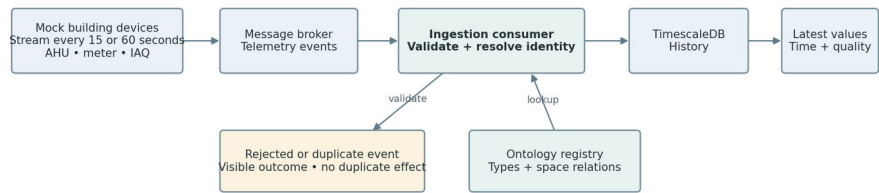
#### Scope and ownership

Required work is evaluated. Bonus work cannot compensate for an unreliable core. Physical gateways, production cloud deployment, equipment control, work orders, 3D visualization, and autonomous rule activation are outside the required scope. You choose and defend the architecture, data stores, service boundaries, APIs, rule representation, user experience, and important trade-offs.

# Task 1 Foundation

## Edge Simulation and Ontology Infra

**Goal:** Create the reliable data foundation needed by the AFDD engine and dashboard.



*Mock building devices continuously stream readings into ontology-linked current and historical data.*

## Required technology boundary

| Layer                       | Required direction                                                                                                                         |
|-----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| Backend                     | Python with FastAPI or Django                                                                                                              |
| Telemetry history           | TimescaleDB                                                                                                                                |
| Relational application data | PostgreSQL or Supabase-managed PostgreSQL when used; it may share TimescaleDB's PostgreSQL foundation                                      |
| Ontology relationships      | Candidate-designed: relational/recursive queries, a PostgreSQL graph extension, or a dedicated graph/RDF database such as Neo4j or GraphDB |

## Build

- A configurable mock-device simulator that continuously streams the supplied AHU, meter, and IAQ readings to the platform, as real devices would. Support a selectable interval such as every 15 or 60 seconds, an event broker, and a separate ingestion consumer. Preserve event IDs and device-recorded timestamps; validate identity, type, timestamp, unit, and quality.
- An ontology registry for containment, installation, direct AHU-to-zone service, point ownership, and measurement scope. Resolve every incoming point ID against it.
- Current-state and TimescaleDB history linked to canonical point identities. Duplicates must have no second effect, and an older observation must not replace a newer current value.
- APIs for discovery, relationships, latest values, history, and health, with structured logs and explicit retry and acknowledgement behavior.

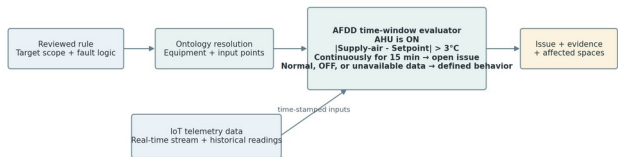
## Demonstrate

- Start and seed the platform with one documented command, then stream simulated readings for Buildings A-C concurrently or by selection. Keep the simulator running so Task 2 can evaluate new data continuously.
- Retrieve current value, history, and spatial context for one AHU, electricity meter, and IAQ device.
- Show duplicate idempotency, visible invalid-event rejection, and missing data that remains visibly missing.

You decide: choose the broker, ontology representation, boundaries, and APIs. Use relational storage, add graph capability, or propose a graph-first model; defend query behavior, operational complexity, migration, new-building onboarding, and hybrid-cloud evolution.

# Task 2 Configurable AFDD engine

**Goal:** Turn one shared engineering rule into spatially targeted, locally configurable monitoring with reproducible evidence.



## Two configurable parts of every rule

| Configuration area | What it controls                                          | Examples                                                                          |
|--------------------|-----------------------------------------------------------|-----------------------------------------------------------------------------------|
| 1 Target scope     | Which entities the rule applies to                        | Property type; buildings; served floor, zone, or room; equipment type; exclusions |
| 2 Fault logic      | How selected equipment and points are evaluated over time | Operating state; comparison; threshold and unit; duration; severity; recovery     |

## Required implementation case

Target AHUs serving tenant areas on selected office-building floors. While an AHU is ON and required readings are recent enough to trust, create one Critical issue when supply-air temperature differs from its setpoint by more than 3°C continuously for 15 minutes. Support one property-specific threshold or duration override; exclude equipment missing a required point.

## Build

- A rule schema or small domain-specific language that stores target scope separately from fault logic. The required case must be configuration, not a hard-coded code path.
- Ontology-based scope resolution with a preview of matched equipment, relationship paths, effective local values, missing points, and other exclusions.
- A scheduler and deterministic evaluator that uses the timestamp recorded by the device simulator, rather than the platform receipt time. The comparison must remain true for 15 continuous minutes, and old readings do not count.
- Documented behavior when an AHU turns OFF, an input is missing or old, readings return to normal, the fault happens again later, or the rule changes. Preserve the rule version used for each issue.
- An issue record preserving the opening rule version, trigger interval, observations, calculated difference, threshold, data quality, and affected zone and rooms.

## Demonstrate

Change target scope independently from fault logic. Show that normal data does not trigger; a fault must last for the full duration; OFF, missing, or old readings cannot create a misleading issue; recovery ends it; a later fault can recur; overrides change the result; and equipment without required points is excluded.

*Possible live-review variation: For Floor 2 AHUs in Buildings B and C, compare return-air temperature with room IAQ temperature in the same zone while ON. Above 5°C creates a Warning. Ask rather than guess how to combine two room readings, how recent data must be, and how long the condition must last.*

You decide: choose rule representation, persistence, and scaling. Explain how new selectors, datapoints, comparisons, and rule families can be added without rebuilding the platform.

# Task 3 Operation Dashboard

**Goal:** Help an operator understand whether the platform is healthy, why an issue occurred, which spaces may be affected, and how the rule behaved.

## Build three focused views

| View                            | Minimum outcome                                                                                                                                                                                     |
|---------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Portfolio and property overview | Navigate property, floor, and zone; see fresh AHU, meter, and IAQ values; active issues; ingestion/evaluation times; and visible errors or lag                                                      |
| Issue investigation             | See the exact rule version, state and timestamps, supply-air temperature (SAT) and setpoint trends, run status, calculated difference, threshold, qualifying interval, data gaps, and trigger point |
| Rule detail and preview         | Read the plain-language intent and structured configuration; inspect selectors, matched assets, exclusions, local overrides, and versions; validate, preview, activate, disable, and adjust         |

## Explainability requirements

- Show the direct AHU-to-zone-to-room served-space path and label it as potentially affected scope. Keep the AHU installation room separate from the spaces it serves.
- Clearly distinguish the supply-air temperature and setpoint measurements that triggered the issue from contextual return-air temperature, AHU alarm, room IAQ, and floor-meter data.
- Always show units, observed time, freshness, and quality. Include loading, empty, error, and insufficient-data states.
- Use React and TypeScript with the Python APIs and persistent data. The views may be separate routes or one coherent drill-down experience.

## Demonstrate

Start from the portfolio overview, open a seeded issue, reconstruct exactly why and when it triggered, identify the potentially affected rooms, review the active rule and local override, and show whether the underlying data pipeline is healthy.

## You decide

Choose the information architecture and interaction design. Measure one investigation outcome, such as whether a reviewer identifies the correct trigger and affected rooms, or how many steps the investigation requires. Explain how you would validate it with property engineers.

## Bonus Historical backtesting

Run a draft rule over a selected historical period using the same evaluator. Show predicted issue and insufficient-data intervals, compare configurations, and keep the backtest isolated from live issues and activation.

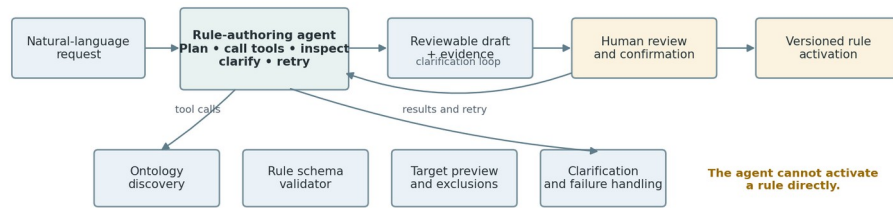
## Task 4 AI-assisted AFDD rule creation

### A path toward a self-evolving AFDD system

A future system could combine operating history with technician know-how to propose rule improvements. Engineers must still review evidence, approve changes, version rules, and roll back safely.

**Goal:** Build a small agentic rule-authoring workflow that turns varied natural-language requests into safe, ontology-aware rule drafts. A single prompt-to-API call is not sufficient.

*Example request: For all office properties, monitor AHUs serving tenant areas. While an AHU is ON, if supply-air temperature differs from its setpoint by more than 3°C continuously for 15 minutes, create a Critical issue.*



*The agent can plan, call bounded tools, inspect results, clarify, and retry. Only a human-confirmed draft can be activated.*

### Build the agentic harness

- An orchestrator that keeps request state and uses bounded tools for ontology discovery, rule validation, target preview, exclusions, and clarification. It may iterate until the draft is valid or stop safely.
- Server-resolved identities and structured rule output. The model must not invent assets, execute code, query unrestricted data, or activate a rule directly.
- Handling for paraphrases, missing details, unexpected ontology results, unsupported logic, empty targets, invalid tool output, timeouts, and retry or stop limits.
- Persistence for the request, tool results, reviewed draft, human confirmation, activation result, audit history, and rule version.

### Evaluate and demonstrate

- Use a repeatable case matrix to measure correct rule structure, target accuracy, required clarification, safe rejection, and zero activation without confirmation.
- Use at least one real model call for the supplied supported, paraphrased, ambiguous, unsupported, no-match, invented-asset, changed-ontology, and recoverable-failure cases. Tests may use deterministic stubs.
- Preserve tool traces, latency, retries, stop reasons, and model or schema versions so failures can be reproduced and improved.

### Bonus MCP access for external AI agents

Expose ontology, AFDD rules, current and historical issues, and evidence through a Model Context Protocol (MCP) server. Other AI systems could then investigate recurring alerts and recommend tuned or new rules. Keep every proposal behind validation, versioning, audit, preview, and human activation. [3]

## 4 Delivery and documentation

---

**Goal:** Deliver a reproducible product that another engineer can start, inspect, test, and demonstrate without your assistance.

### Submission route

Submit the complete assessment in one private GitHub repository. Grant repository access only to these reviewer accounts: thakorneyp11, KornCode, toss911, ZayarNaing, andamanopal, and jayjirayut. Keep the repository private and include the application, infrastructure, tests, seed data, and every required document in the repository.

### Deliverables

| # | Deliverable                    | Required content                                                                                                | Reviewer check                                                      |
|---|--------------------------------|-----------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------|
| 1 | Working prototype              | Docker Compose for UI, API, broker, telemetry simulator, ingestion, AFDD worker, and stores                     | One documented command starts healthy services                      |
| 2 | Reproducible data              | Migrations plus seed, reset, and telemetry simulation or fixture replay commands                                | Seed produces the expected inventory and supplied cases             |
| 3 | Private GitHub repository      | Code, Dockerfiles, tests, lockfiles, environment template, diagrams, and required documents; no secrets         | Only the named reviewer accounts have access; core tests pass       |
| 4 | API contract                   | OpenAPI or equivalent documentation for discovery, telemetry, rules, issues, and health                         | Reviewer can inspect and call the core endpoints                    |
| 5 | README and technical documents | README.md plus architecture, decisions, limitations, and demonstration guide                                    | Reviewer can find, run, and explain the product from the repository |
| 6 | Demo evidence                  | Representative screenshots and either a short demonstration video or a clearly sequenced screenshot walkthrough | Reviewer can understand the working journey before the live review  |

## Delivery documentation checklist

---

Keep these materials in the same private GitHub repository. Markdown is acceptable; diagrams may be embedded or linked from a clearly named docs folder.

| Document                 | Required contents                                                                                                                                                                                                                                                                                                                                                      |
|--------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| README.md                | Product purpose, implemented scope, repository and document index, architecture summary, and known limitations. Include prerequisites and a quickstart for configuration, environment variables, setup, migrations, seed/reset, telemetry simulation or replay, startup, health checks, tests, API access, and demonstration.                                          |
| System architecture      | Component diagram and the main telemetry ingestion, AFDD evaluation, and natural-language rule flows. Show service boundaries, data stores, and important trust boundaries.                                                                                                                                                                                            |
| Brickschema model        | Entity classes, relationships, application metadata, ontology-storage approach, links to time-series identities, and how a new building or changed ontology is onboarded.                                                                                                                                                                                              |
| AFDD behavior            | Rule schema, target-scope and fault-logic separation, timing assumptions, issue lifecycle, evidence model, and supported limitations.                                                                                                                                                                                                                                  |
| AI rule-authoring design | Agent, bounded tools, deterministic services, stores, and trust boundaries. Include sequence diagrams for a successful request and one clarification or failure path; tool contracts; state, retry, stopping, and observability decisions; and an evolution plan for ontology/schema versioning, compatibility, testing, cache refresh, and scaling across more sites. |
| Technical decisions      | Important alternatives and trade-offs, including data stores, ontology representation, service boundaries, failure handling, performance, and scalability.                                                                                                                                                                                                             |
| Demonstration guide      | End-to-end sequence, screenshots and demo-video link or screenshot walkthrough, expected seeded outcomes, plus a short AI-assistance note naming tools used and one suggestion corrected or rejected.                                                                                                                                                                  |

## 5 Evaluation and review

We evaluate the required product journey, the reasoning behind your decisions, and your ability to explain and adapt the system. Bonuses strengthen the discussion but do not compensate for an unreliable core.

### Evaluation weights

| Area                      | Weight | What we look for                                                                                       |
|---------------------------|--------|--------------------------------------------------------------------------------------------------------|
| Foundation                | 20%    | Reliable event flow, semantic identity, data correctness, and traceability                             |
| Configurable AFDD engine  | 30%    | Ontology targeting, deterministic evaluation, configuration, evidence, and lifecycle                   |
| Operation Dashboard       | 20%    | Clear system health, issue explainability, affected-space reasoning, and usable analysis               |
| AI-assisted rule creation | 15%    | Useful interpretation, safe validation, clarification, preview, confirmation, and evaluation           |
| Technical ownership       | 15%    | Product judgment, architecture, testing, observability, reproducibility, trade-offs, and communication |

### Core completion gates

- Docker Compose starts, seeds, and replays the supplied cases reproducibly.
- The evaluator matches the supplied targeting, timing, and lifecycle outcomes.
- The dashboard reconstructs a seeded issue and its potentially affected spaces.
- Natural-language input cannot bypass validation, ontology resolution, preview, or confirmation.

### What strong senior ownership looks like

- Define the product outcome first and make deliberate scope and architecture decisions.
- Separate domain truth from implementation choice and make assumptions visible.
- Design for failure, traceability, performance, change, and operation by another engineer.
- Use AI critically: verify generated work, reject unsafe shortcuts, and explain the result.

### Review session

In a 60-75 minute review, demonstrate the hero journey, trace one event through the system, investigate one failure, and explain your trade-offs. We may introduce one small in-scope variation and ask how you would adapt the design. We value your reasoning more than presentation polish.

### References

- [1] Brickschema introduction: <https://docs.brickschema.org/intro.html>
- [2] Brickschema relationships: <https://docs.brickschema.org/brick/relationships.html>
- [3] Model Context Protocol architecture: <https://modelcontextprotocol.io/docs/learn/architecture>